

PROMPT — Chatbot de Ayuda del Sistema

Pegar completo en Replit

Necesito agregar un botón de ayuda flotante que aparezca en todas las páginas del sistema. Al hacer clic abre un chat lateral donde el staff puede hacer preguntas sobre cómo usar el sistema y recibe respuestas automáticas basadas en el manual interno.

PARTE 1 — Endpoint en `server/routes.ts`

Agregar el siguiente endpoint que usa la integración OpenAI ya existente en el proyecto:

```
app.post("/api/help/chat", requireAuth, async (req, res) => {
  try {
    const { message, history } = req.body;
    // history es un array de { role: "user" | "assistant", content: string }
    // con los últimos mensajes de la conversación (máximo 10)

    const SYSTEM_PROMPT = `[PEGAR AQUÍ EL CONTENIDO COMPLETO DEL ARCHIVO MARAN_Ma

const messages = [
  { role: "system", content: SYSTEM_PROMPT },
  ...(history || []).slice(-10),
  { role: "user", content: message }
];

const completion = await openai.chat.completions.create({
  model: "gpt-4o-mini",
  messages,
  max_tokens: 500,
  temperature: 0.3,
});

const reply = completion.choices[0].message.content;
res.json({ reply });
} catch (error) {
  res.status(500).json({ error: "Error al procesar la consulta" });
}
});
```

Importante: El contenido del `SYSTEM_PROMPT` es el texto completo del manual del sistema. Está en el archivo `MARAN_Manual_Sistema_Chatbot.md` en el proyecto. Leer ese archivo e insertar su contenido completo como string en el `SYSTEM_PROMPT`.

Si el archivo no está en el proyecto, crear `server/help-manual.ts` con el contenido del manual exportado como constante:

```
export const HELP_MANUAL = `[contenido del manual aquí]`;
```

Y en el endpoint importarlo:

```
import { HELP_MANUAL } from "../help-manual";
// ...
const SYSTEM_PROMPT = HELP_MANUAL;
```

PARTE 2 — Componente del chat de ayuda

Crear `client/src/components/help-chat.tsx`:

```
// Componente con:
// - Botón flotante circular en la esquina inferior derecha con ícono HelpCircle
//   Color: azul primario del sistema
//   Siempre visible en todas las páginas
//
// - Panel lateral que se abre al hacer clic en el botón
//   Ancho: 380px en desktop, full width en mobile
//   Se abre desde la derecha con animación suave
//
// - Header del panel:
//   Título: "Ayuda — Maran Suite"
//   Subtítulo: "Preguntame cómo usar el sistema"
//   Botón X para cerrar
//
// - Área de mensajes:
//   Scroll automático al último mensaje
//   Mensajes del usuario alineados a la derecha (fondo azul, texto blanco)
//   Mensajes del asistente alineados a la izquierda (fondo gris claro)
//   Indicador de "escribiendo..." (tres puntos animados) mientras espera respues
//
// - Mensaje inicial automático al abrir por primera vez:
//   "Hola 🙌 Soy el asistente de Maran Suite. Podés preguntarme cómo hacer
```

```
// cualquier cosa en el sistema: reservas, check-in, caja, reportes, y más."
//
// - Input de texto en el footer:
// Placeholder: "Escribí tu pregunta..."
// Botón enviar (ícono Send)
// Enviar también con Enter
// Deshabilitar input mientras espera respuesta
//
// - El historial de la conversación se mantiene mientras la sesión está activa
// Al cerrar y reabrir el panel, los mensajes anteriores siguen visibles
// Botón "Nueva consulta" en el header para limpiar el historial
```

Implementación del fetch:

```
const sendMessage = async (userMessage: string) => {
  setIsLoading(true);
  try {
    const res = await fetch("/api/help/chat", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      credentials: "include",
      body: JSON.stringify({
        message: userMessage,
        history: messages.slice(-10).map(m => ({
          role: m.role,
          content: m.content
        })))
    });
    const data = await res.json();
    // agregar data.reply a los mensajes
  } catch (error) {
    // mostrar mensaje de error amigable
  } finally {
    setIsLoading(false);
  }
};
```

PARTE 3 — Integrar en el layout principal

En `client/src/App.tsx`, dentro del componente `AuthenticatedApp` (el que se muestra cuando el usuario está logueado), agregar el componente `HelpChat` al final del JSX, fuera del router pero dentro del layout:

```
import HelpChat from "@components/help-chat";

// Dentro del return de AuthenticatedApp:
return (
  <AuthContext.Provider value={{ user, logout: handleLogout }}>
    { /* ... layout existente ... */ }
    <HelpChat /> { /* ← agregar esto al final, antes del cierre */ }
  </AuthContext.Provider>
);
```

Esto hace que el botón flotante aparezca en todas las páginas del sistema automáticamente.

Notas para Replit

- Usar `gpt-4o-mini` como modelo — es más rápido y económico para respuestas de ayuda
- `temperature: 0.3` para respuestas consistentes y precisas
- El panel de ayuda NO debe interferir con ningún otro elemento de la interfaz
- El botón flotante debe tener z-index alto para que siempre esté visible por encima del contenido
- En mobile el panel ocupa el ancho completo de la pantalla
- No mostrar el botón de ayuda en la página de login
- Verificar que compila sin errores TypeScript al finalizar